

Class Inheritance, Overriding and Statics

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

Inheritance with `extends`

- `extends` reuses another class's behavior
- `super(...)` calls the parent constructor

```
class Animal {  
  constructor(name) { this.name = name; }  
  speak() { console.log(this.name + " makes a sound."); }  
}  
  
class Dog extends Animal {  
  constructor(name, breed) {  
    super(name);          // must come FIRST  
    this.breed = breed;  
  }  
}  
  
new Dog("Rex", "Lab").speak(); // "Rex makes a sound."
```

`super()` Must Come First

```
class Dog extends Animal {  
  constructor(name, breed) {  
    // this.breed = breed;    // ReferenceError!  
    super(name);  
    this.breed = breed;      // fine, after super  
  }  
}
```

- Until `super()` runs, `this` does not exist yet
- If a subclass has no constructor at all, JS supplies one that just forwards every argument to `super`

Overriding a Method

- Define a method with the same name to replace the parent's

```
class Dog extends Animal {  
  speak() { console.log(this.name + " barks!"); }  
}  
  
new Dog("Rex").speak(); // "Rex barks!"
```

- Lookup walks **up** the chain and stops at the first match
- The parent's version is not deleted, just shadowed

Calling Back Up With `super.method()`

- Extend the parent's behavior instead of replacing it

```
class Dog extends Animal {  
  speak() {  
    console.log("Clearing throat...");  
    super.speak();           // run the parent's version too  
  }  
}  
  
new Dog("Rex").speak();  
// "Clearing throat..."  
// "Rex makes a sound."
```

Static Members

- `static` belongs to the **class**, not to any instance

```
class Vehicle {  
  static registry = "State DMV";  
  static count = 0;  
  
  constructor(make) {  
    this.make = make;  
    Vehicle.count++;  
  }  
}  
  
new Vehicle("Toyota");  
console.log(Vehicle.registry); // "State DMV"  
console.log(Vehicle.count);    // 1
```

Statics Are Not on Instances

```
const v = new Vehicle("Mazda");

console.log(Vehicle.registry); // "State DMV"
console.log(v.registry);       // undefined
```

- Read them off the **class name**, always
- A subclass's `super()` still runs the parent constructor, so a shared counter counts subclass instances too

Static Factory Methods

- A named alternative to `new` with magic arguments

```
class Car extends Vehicle {  
  constructor(make, doors) { super(make); this.doors = doors; }  
  
  static coupe(make) { return new Car(make, 2); }  
}  
  
const c = Car.coupe("Mazda");  
console.log(c.doors); // 2
```

- `Car.coupe("Mazda")` reads better than `new Car("Mazda", 2)`

instanceof Walks the Chain

```
class Bicycle {}

const car = new Car("Toyota", 4);

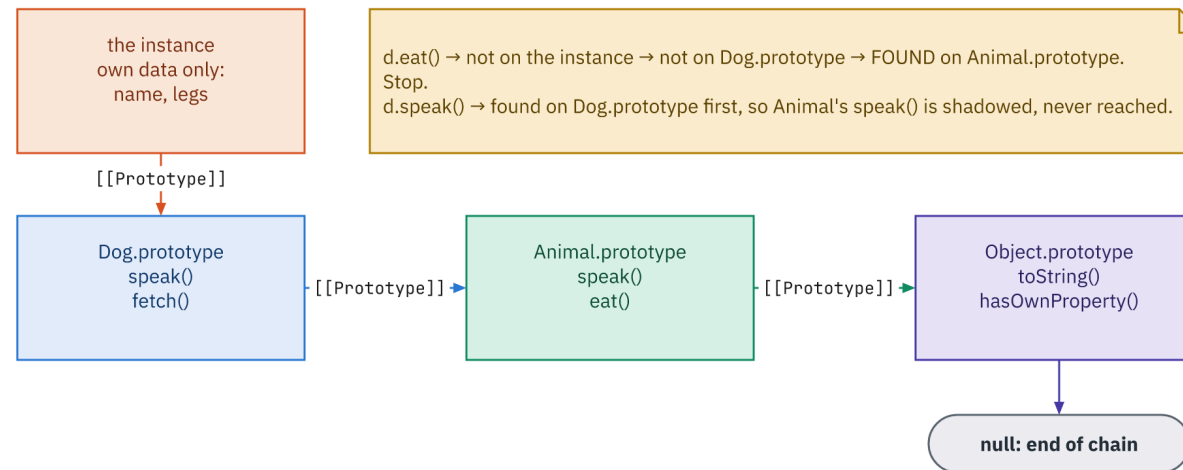
car instanceof Car;    // true
car instanceof Vehicle; // true - up the chain
car instanceof Object; // true - everything ends here
car instanceof Bicycle; // false
```

- `instanceof` asks "is this constructor's prototype anywhere in my chain?"

What **extends** Builds

The Prototype Chain

Property lookup walks UP the chain and stops at the first match. class builds exactly this.



class Dog extends Animal builds this chain for you: extends sets the link, super() runs the parent constructor, and instanceof asks whether a given prototype appears anywhere along the chain.

- Lookup walks **up** until it finds a match, then stops
- **super()** runs the parent constructor before you touch **this**